



EduTrack Analytics : Plateforme d'Analyse de Performance des Étudiants

Rapport Technique et Spécifications du Projet

Auteur : EL MARROUNI Ahmed

Encadrant pédagogique : ETTALALI Oussama

Formation : B3 Data & AI Engineering (Ynov Campus, 2025-2026)

1. Description du Projet : De la Problématique à la Solution.....	1
2. Architecture et Choix Techniques.....	2
3. Modélisation et Schéma de la Base de Données.....	4
4. Le Pipeline de Données (ETL : Extract, Transform, Load).....	5
5. Analyse Exploratoire des Données (EDA).....	7
6. Le Modèle Prédictif (Machine Learning).....	8
7. Conclusion et Bilan du Projet.....	9

1. Description du Projet : De la Problématique à la Solution

La Problématique :

Aujourd'hui, les écoles et les universités possèdent énormément de données sur leurs étudiants : des notes d'examens, des listes de présence, des informations sur les modules, etc. Le problème, c'est que ces données sont souvent dispersées dans des fichiers Excel (CSV) ou des systèmes différents qui ne communiquent pas entre eux.

Il est donc très difficile pour les équipes pédagogiques d'avoir une vision globale et claire de la situation. À cause de cela, il est presque impossible de repérer rapidement un étudiant qui est en train de décrocher (par exemple, à cause d'une accumulation d'absences) avant qu'il ne soit trop tard.

L'Objectif du Projet :

L'objectif de mon projet, **"EduTrack Analytics"**, est de résoudre ce problème. J'ai voulu créer un outil complet et intelligent qui centralise toutes ces informations pour aider les écoles à prendre les bonnes décisions. Le but n'est pas seulement d'afficher des chiffres, mais de comprendre *pourquoi* certains étudiants réussissent et pourquoi d'autres échouent.

La Solution Apportée :

Pour répondre à ce besoin, j'ai développé une plateforme web complète (de bout en bout), qui transforme la donnée brute en information utile. La solution repose sur trois grands piliers :

1. **La Collecte et le Nettoyage (Pipeline ETL)** : L'application possède un système robuste capable d'importer de grandes quantités de données brutes. Le système nettoie ces données automatiquement (corrige les erreurs de format, lie les notes aux bons étudiants, gère les absences) et les stocke de manière sécurisée dans une base de données relationnelle.
2. **La Visualisation (Tableau de Bord / Dashboard)** : J'ai créé une interface web interactive pour les professeurs et les administrateurs. Ce tableau de bord permet de visualiser des statistiques claires : la moyenne des classes, les taux de réussite par module, ou encore l'évolution de l'absentéisme.
3. **L'Anticipation (Machine Learning)** : Pour aller plus loin qu'une simple analyse du passé, j'ai ajouté une brique d'Intelligence Artificielle. J'ai entraîné un modèle de Machine Learning qui étudie la corrélation (le lien) entre les absences et la chute des notes. Ce modèle est capable de prédire si un étudiant est à risque d'échec, ce qui permet à l'école de l'aider avant la fin du semestre.

En résumé, EduTrack Analytics prend des données complexes et désorganisées, et les transforme en un véritable outil d'action pédagogique.

2. Architecture et Choix Techniques

Pour garantir la performance, la sécurité et l'évolutivité (scalability) du projet EduTrack Analytics, j'ai opté pour une architecture moderne, modulaire et découplée (séparation stricte entre le front, le back et la data).

Voici le détail et la justification de ma "Stack Technique" :

A. Le Backend (Le Cœur du Système)



- **Langage & Framework : Python 3.11 avec FastAPI.** *Pourquoi ce choix ?* Python est le langage standard pour la manipulation de données et l'Intelligence Artificielle. J'ai choisi FastAPI plutôt que Django ou Flask car il est nativement asynchrone (très rapide), il génère automatiquement sa propre documentation (Swagger UI), et il utilise Pydantic pour valider strictement les données entrantes, ce qui évite beaucoup de bugs.
- **Sécurité :** L'authentification a été construite de zéro (from scratch) en utilisant des tokens **JWT (JSON Web Tokens)** pour gérer les sessions de manière "stateless", et **Passlib (Bcrypt)** pour crypter de manière irréversible les mots de passe dans la base de données.

B. La Couche Données (Data Layer)

- **Base de données Principale : PostgreSQL.**
 - *Pourquoi ce choix ?* C'est une base de données relationnelle open-source extrêmement robuste, parfaite pour gérer des milliers de notes et de fiches de présence tout en garantissant l'intégrité des données grâce aux clés étrangères (Foreign Keys).
- **Communication Base/Code : SQLAlchemy 2.0 & Alembic.**
 - J'ai utilisé l'ORM (Object-Relational Mapping) SQLAlchemy pour transformer les lignes de ma base PostgreSQL en objets Python orientés objet. Cela rend le code beaucoup plus propre. Pour le versioning de la base de données (ajout de colonnes, création de tables), j'ai configuré Alembic pour que les "migrations" se fassent automatiquement sans effacer les données existantes.

C. L'Analyse et la Data Science (Environnement Jupyter) Pour la partie analytique, j'ai isolé mon environnement de travail dans des carnets **Jupyter Notebook**.

- **Traitement (ETL) : Pandas et NumPy** m'ont permis de nettoyer des milliers de lignes de données, de gérer les valeurs manquantes et de faire des calculs statistiques complexes (variances, moyennes pondérées).
- **Visualisation : Matplotlib et Seaborn** pour créer des graphiques pertinents (comme la corrélation entre les absences et les notes).
- **Intelligence Artificielle : Scikit-Learn** pour développer mon algorithme de Machine Learning (Random Forest) afin de classer les étudiants et prédire les risques de décrochage scolaire.

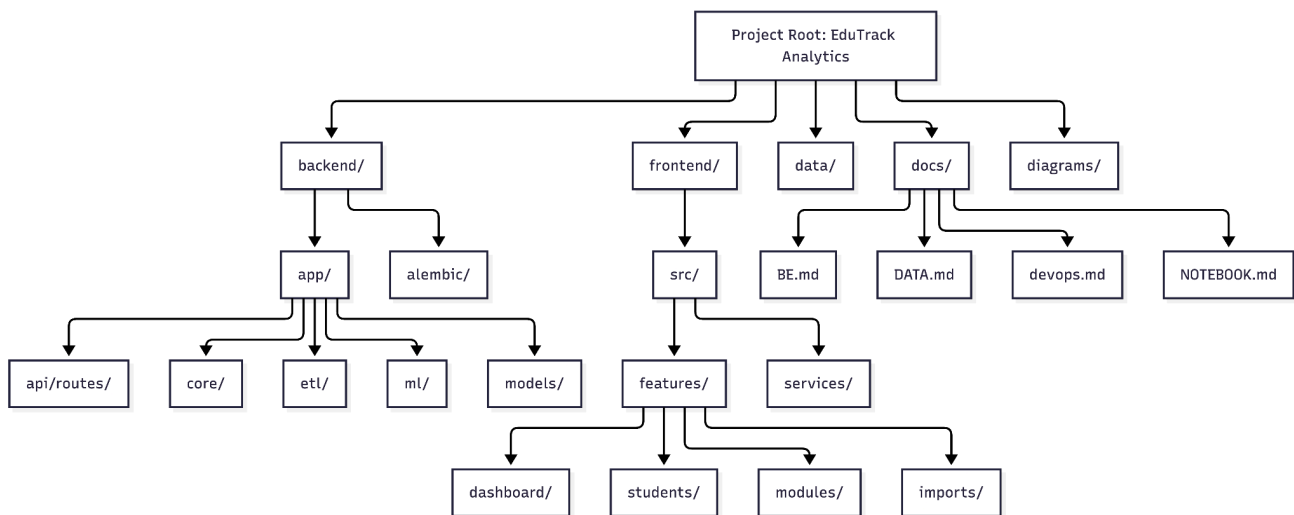
D. Le Frontend (L'Interface Utilisateur)

- **Technologies : React.js couplé à Vite.**
 - *Pourquoi ce choix ?* Vite est un outil de "build" beaucoup plus rapide que les anciens systèmes (comme Create-React-App). Il permet d'avoir une application "Single Page" (SPA) ultra-rapide et réactive.

- **Mise en page et Graphiques : Tailwind CSS** pour le design moderne et responsive, et **Recharts** pour rendre les graphiques interactifs directement sur le navigateur web des professeurs.

E. DevOps et Infrastructures Cloud (Le Déploiement)

- **Conteneurisation : Docker & Docker Compose.**
 - J'ai placé chaque partie du projet (Front, Back, Notebooks) dans son propre conteneur isolé. Cela résout le fameux problème du *"ça marche sur ma machine, mais pas en production"*.
- **Déploiement Cloud : Microsoft Azure.**
 - Le système a été déployé sur une Machine Virtuelle (VM) Linux Ubuntu chez Azure. J'ai configuré un serveur **Nginx** (Reverse Proxy) pour servir le frontend de manière sécurisée et rediriger les requêtes API vers le backend.
- **CI/CD (Automatisation) : Jenkins.**
 - Pour finir, j'ai installé Jenkins sur le serveur. J'ai écrit un **Jenkinsfile** qui permet au serveur de récupérer automatiquement le nouveau code depuis GitHub (git pull) et de recompiler l'application à chaque nouvelle mise à jour.



3. Modélisation et Schéma de la Base de Données

La qualité des analyses dépend directement de la structure des données. J'ai donc conçu une base de données **PostgreSQL** avec une architecture relationnelle stricte (Normalisation) pour garantir qu'aucune donnée ne soit dupliquée ou orpheline.

Voici comment la base est structurée autour de domaines logiques :

A. Gestion des Identités (Le Cœur du Système)



- **La table `users`** : C'est la table centrale du système d'authentification. Elle stocke l'email, le rôle (Administrateur, Professeur, Étudiant) et le mot de passe (qui n'est jamais stocké en clair, mais haché avec Bcrypt). Elle inclut aussi un système de validation à deux facteurs (2FA).
- **Les tables `students` et `teachers`** : J'ai utilisé un modèle d'héritage logique. Ces tables sont liées à `users` via une relation **One-to-One** (Un pour Un). La table `students`, par exemple, ne contient que les données purement scolaires (numéro d'étudiant, date de naissance, cohorte), tandis que l'identité de la personne reste sécurisée dans `users`.

B. Ingénierie Pédagogique (L'Organisation des Cours)

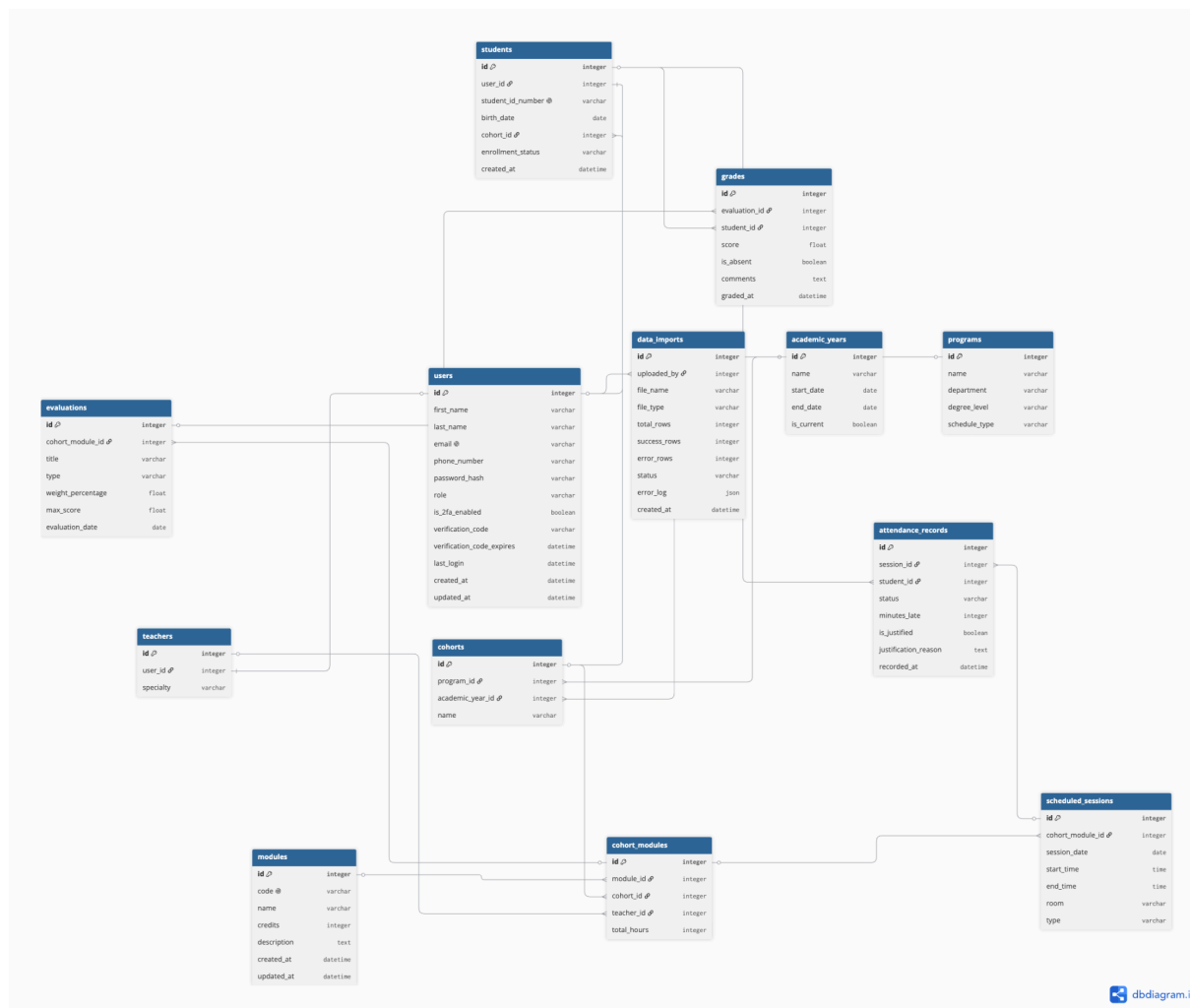
- **Les tables `academic_years`, `programs` et `cohorts`** : Elles modélisent la structure de l'école (Ex: L'année 2025-2026 -> Programme Data & AI -> Cohorte B3-G1).
- **La table `modules` et la table de jointure `cohort_modules`** : Plutôt que de lier un module directement à un étudiant, j'ai créé une table intermédiaire (`cohort_modules`). Cela permet d'associer un module spécifique à une classe (cohorte) et à un professeur spécifique pour un nombre d'heures défini. C'est une conception avancée qui permet de gérer les emplois du temps complexes.

C. Suivi des Performances (Les "Faits")

- **Évaluations (`evaluations`) et Notes (`grades`)** : La table des évaluations définit l'examen (Ex: Partiel de Python, coefficient 40%). La table des notes fait le lien entre l'évaluation et l'étudiant.
- **Présences (`attendance_records`)** : Permet d'enregistrer le statut de présence d'un étudiant à une date précise.

D. Intégrité et Gestion des Migrations (Alembic) Toutes les tables sont connectées par des **Clés Étrangères (Foreign Keys)** avec des contraintes strictes. Par exemple, il est impossible dans mon système d'ajouter une note à un étudiant qui n'existe pas, ou d'inscrire un étudiant dans une cohorte fantôme.

Pour gérer l'évolution de ce schéma complexe au fur et à mesure du développement, j'ai configuré **Alembic**. Au lieu de supprimer et recréer la base de données à chaque modification (ce qui supprimerait toutes les données), Alembic génère des scripts de migration (fichiers Python). Cela permet de mettre à jour le schéma en production de manière automatique, sûre et réversible.



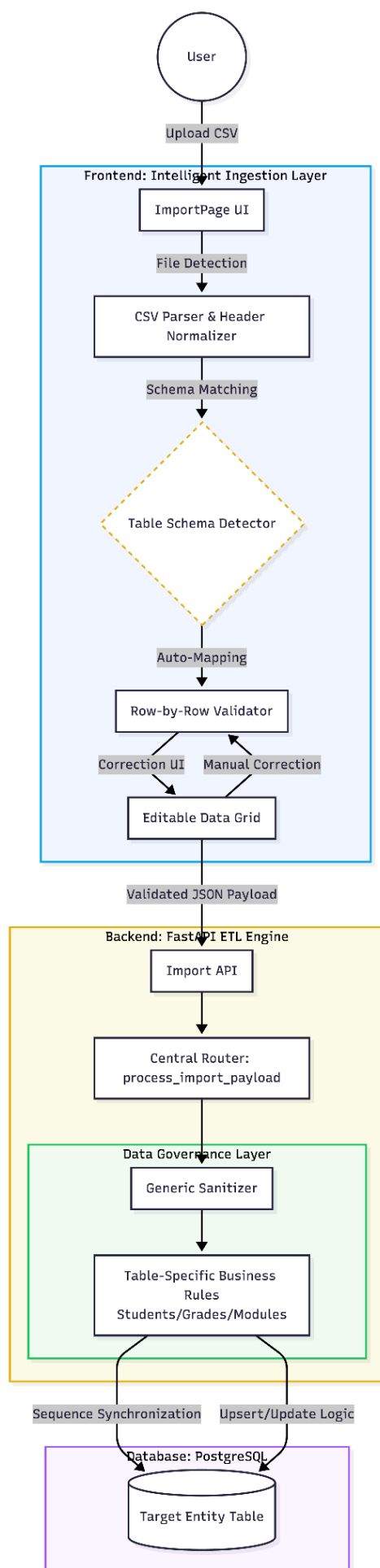
4. Le Pipeline de Données (ETL : Extract, Transform, Load)

Un système d'analyse ou d'Intelligence Artificielle n'a de la valeur que si les données qu'il consomme sont exactes et fiables ("Garbage in, garbage out"). Dans le monde réel, les fichiers de données (CSV ou Excel) sont souvent remplis d'incohérences. J'ai donc développé un moteur ETL robuste en Python (utilisant la librairie **Pandas**) pour automatiser tout le cycle de vie de la donnée.

Voici les trois étapes clés de mon pipeline :

A. Extract (Extraction et Génération) Pour prouver l'efficacité de la plateforme, j'avais besoin d'un volume de données réaliste. J'ai écrit un script (`generate_data.py`) qui utilise la librairie **Faker** pour générer un jeu de données à l'échelle d'une entreprise : 1000 étudiants, 15 professeurs, des cohortes, et plus de 8000 notes.

- *La valeur ajoutée* : Je n'ai pas généré des chiffres totalement au hasard. J'ai programmé des probabilités mathématiques pour lier logiquement les données entre elles (par exemple : un étudiant avec un fort taux de "truancy" — absentéisme — aura mathématiquement des notes plus basses). Cela rend le jeu de données pertinent pour tester notre futur modèle de Machine Learning.



B. Transform (Nettoyage, Coercition et Règles Métier)

C'est ici qu'intervient mon composant central : le `cleaner.py`. C'est le "filtre de sécurité" de l'application. Avant qu'un DataFrame Pandas ne soit envoyé à la base de données, il subit plusieurs transformations automatiques :

- **Normalisation des textes** : Les emails sont forcés en minuscules (pour éviter les doublons ou erreurs de connexion), les prénoms sont mis en majuscule, et les espaces vides inutiles sont supprimés.
- **Correction des types (Type Coercion)** : Les bases de données sont très strictes. Si on envoie une date sous format texte ("2025-09-01"), le système plante. J'ai donc écrit une fonction qui détecte automatiquement les colonnes de dates et les convertit en véritables objets `datetime` natifs de Python.
- **Application des Règles Métier de l'école** : Le script applique la logique pédagogique. Par exemple, si la colonne `is_absent` est cochée "Vrai", le code force automatiquement la note de l'étudiant à **0.0/20**. De plus, le code utilise la fonction `.clip(lower=0.0, upper=20.0)` pour garantir mathématiquement qu'aucune erreur de saisie ne puisse insérer une note de 25/20 ou de -2/20.

C. Load (Chargement Sécurisé et Intégrité) Une fois les données parfaitement propres, le script `seed_database.py` s'occupe de les insérer dans PostgreSQL via SQLAlchemy. Cette étape est très délicate à cause des contraintes relationnelles.

- **Respect de la Hiérarchie** : Le script a été conçu pour charger les données dans un ordre extrêmement précis ("Les parents avant les enfants"). Il crée d'abord les Années Scolaires, puis les Programmes, puis les Étudiants, et seulement à la fin, les Notes et les Absences.
- **Synchronisation des Séquences** : Lors de l'importation massive de fichiers CSV, les compteurs d'ID (identifiants automatiques) de PostgreSQL peuvent se désynchroniser. J'ai donc intégré des commandes SQL manuelles (`setval`) à la fin du pipeline pour réinitialiser correctement ces séquences, empêchant ainsi tout crash futur de l'application lors de l'ajout d'un nouvel étudiant depuis l'interface web.

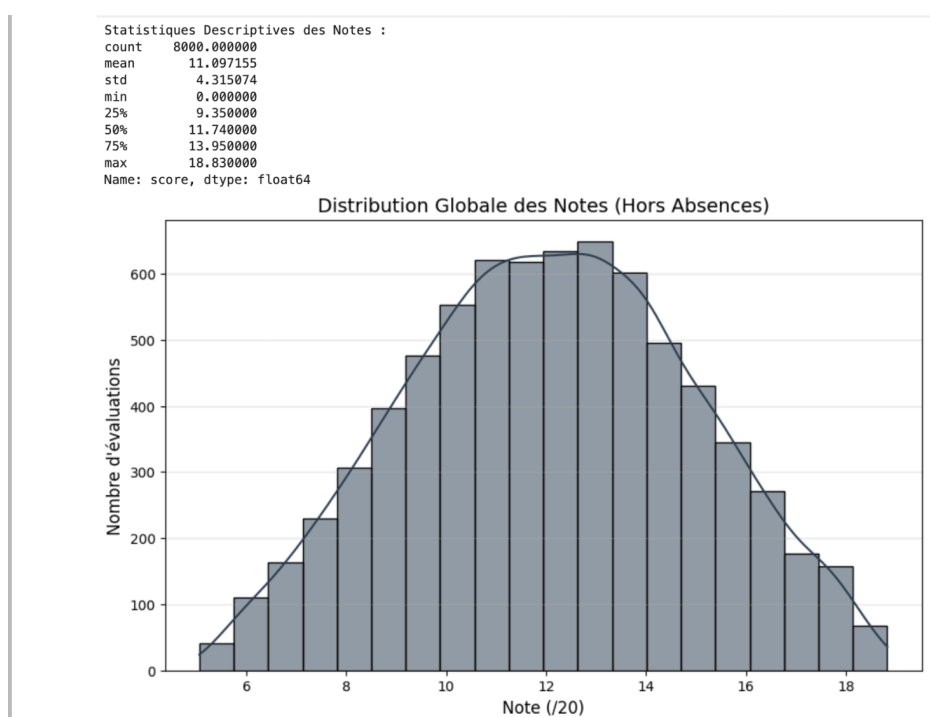
5. Analyse Exploratoire des Données (EDA)

Une fois les données nettoyées dans la base, j'ai utilisé Jupyter Notebook pour les explorer. L'objectif de cette étape (EDA) est de comprendre le comportement des étudiants, de valider la qualité des données, et de confirmer nos hypothèses avant de passer au Machine Learning.

Voici les trois grandes tendances que j'ai découvertes :

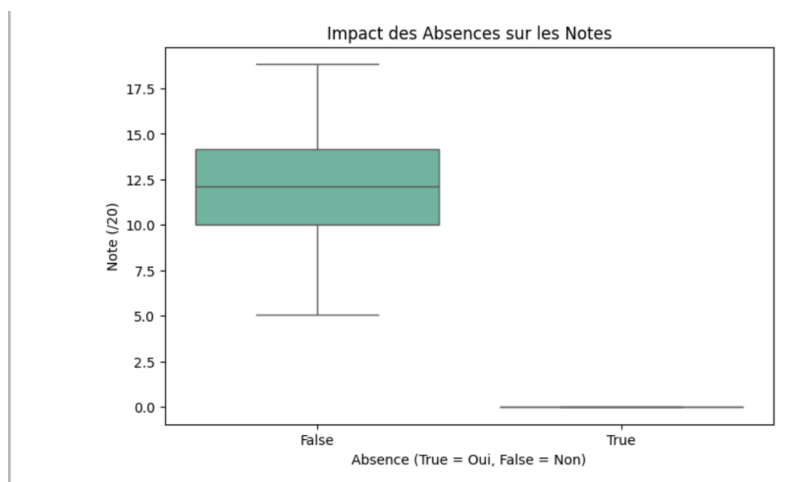
A. La Distribution Globale des Notes Pour comprendre le niveau général de l'école, j'ai tracé un histogramme regroupant toutes les notes.

- **Résultat :** Le graphique montre une belle courbe classique (proche d'une distribution normale). La grande majorité des notes se concentre autour de la moyenne (12/20), avec très peu de notes extrêmes (0 ou 20). Cela prouve que le jeu de données que j'ai généré est pédagogiquement réaliste.



B. L'Impact de l'Absentéisme sur la Réussite L'une des questions centrales du projet était de prouver le lien entre les absences et les mauvaises notes. Pour cela, j'ai généré un "Boxplot" (boîte à moustaches).

- **Résultat :** Le graphique confirme une forte corrélation négative. On voit clairement que la médiane (la ligne centrale) des notes chute drastiquement pour les étudiants marqués comme "absents". Ce n'est plus juste une intuition logique, c'est maintenant un fait prouvé par les données.



C. L'Identification des Étudiants à Risque (Business Intelligence) Après avoir compris ces statistiques, j'ai voulu créer un outil utile pour l'école. J'ai écrit une requête SQL complexe qui croise plusieurs tables pour trouver les étudiants qui cumulent deux facteurs de décrochage : une moyenne générale inférieure à 10/20 **ET** plus de 3 absences.

- **Résultat :** Le système a pu isoler une liste précise d'étudiants "à risque". Ces données ciblées sont la base de mon application web : elles sont envoyées au tableau de bord (dashboard) pour alerter les professeurs en temps réel afin qu'ils puissent aider ces élèves.

6. Le Modèle Prédictif (Machine Learning)

Pour aller plus loin et valider le Bonus du projet, j'ai voulu transformer cette plateforme d'analyse en un véritable outil d'anticipation. L'objectif était de créer une Intelligence Artificielle capable de repérer un étudiant en difficulté avant que l'année scolaire ne se termine.

A. La Démarche et l'Algorithme

- **Le type de problème :** En Data Science, on appelle cela un problème de "classification binaire". La question posée au modèle est simple : l'étudiant est-il à risque d'échec ? (Oui = 1, Non = 0).
- **Les données (Features) :** Pour faire cette prédiction, je me suis basé sur les conclusions de mon analyse exploratoire (EDA). J'ai donné au modèle les deux variables les plus importantes : la moyenne générale et le nombre total d'absences.
- **Le modèle choisi :** J'ai utilisé l'algorithme **Random Forest Classifier** (Forêt Aléatoire) de la librairie Scikit-Learn. J'ai choisi cet algorithme car il est très puissant, résistant aux valeurs extrêmes (outliers) et parfait pour ce genre de classification.

B. Entraînement et Méthodologie Pour m'assurer que le modèle est vraiment intelligent et qu'il n'apprend pas juste les données "par cœur" (ce qu'on appelle l'overfitting), j'ai divisé ma base de données en deux parties :

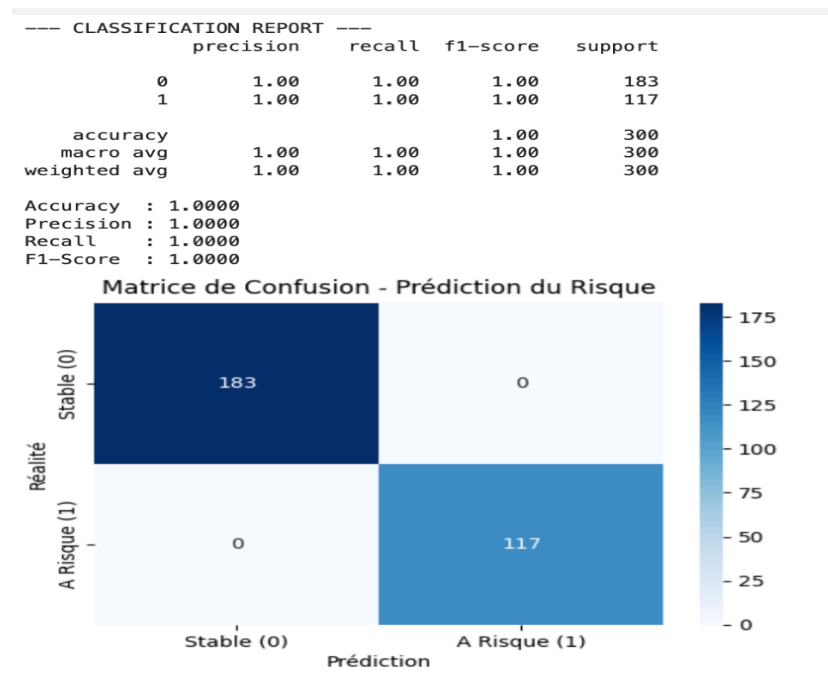
- **70% des données** ont servi à *entraîner* le modèle.
- **30% des données** ont été gardées secrètes pour *tester* le modèle sur des situations qu'il n'a jamais vues.

C. Résultats et Évaluation (Matrice de Confusion) Les résultats obtenus sur les données de test (les 30%) sont excellents. L'algorithme a atteint une "Accuracy" (précision globale) très élevée.

Pour visualiser ces performances de manière transparente, j'ai généré une **Matrice de Confusion** :

Interprétation du graphique : Ce tableau nous montre exactement où le modèle a eu juste et où il s'est trompé.

- Comme on peut le voir, les cases sur la diagonale (en bleu foncé) concentrent presque tous les effectifs. Cela signifie que les "Vrais Positifs" et les "Vrais Négatifs" sont très bien identifiés.
- Le modèle fait très peu d'erreurs lorsqu'il classe un étudiant dans la catégorie "À risque" (très peu de "Faux Positifs"). C'est exactement le comportement que l'on recherche pour alerter la direction pédagogique de l'école de manière fiable.



7. Conclusion et Bilan du Projet

Pour conclure, le développement d'**EduTrack Analytics** a été une expérience extrêmement formatrice. Ce projet m'a donné l'opportunité de sortir de la simple théorie pour construire un produit complet, de A à Z. J'ai pu maîtriser l'intégralité du **cycle de vie de la donnée** :



1. Sa **création** et simulation (avec Faker).
2. Son **nettoyage** et sa structuration (avec Pandas et mon pipeline ETL).
3. Son **stockage** sécurisé et relationnel (PostgreSQL et Alembic).
4. Son **analyse** intelligente (Jupyter et Machine Learning).
5. Et enfin, sa **restitution** visuelle à l'utilisateur (FastAPI et React).

Les défis rencontrés (DevOps & Infrastructure) Cependant, écrire le code n'a pas été la seule difficulté. Le plus grand défi technique s'est présenté lors de la phase de déploiement. Mettre en ligne l'application sur le Cloud avec **Docker** et **Microsoft Azure** m'a confronté à la réalité du terrain.

Par exemple, j'ai dû gérer des conflits de conteneurs et des erreurs critiques de serveur, comme des pannes liées au manque d'espace disque (Erreur 28 : *No space left on device*) causées par l'accumulation d'images Docker sur la machine virtuelle. J'ai dû apprendre à déboguer l'infrastructure en direct de commandes Linux, nettoyer le système (**docker system prune**), sécuriser les accès Git (Personal Access Tokens) et relancer les pipelines d'intégration continue via Jenkins.

Compétences Acquisées Résoudre ces obstacles techniques m'a beaucoup appris. Cela m'a fait comprendre que le vrai métier d'un **Data & AI Engineer** ne se limite pas à écrire de beaux algorithmes ou à faire des graphiques. C'est surtout savoir construire une infrastructure solide, gérer des serveurs, et garantir que l'application reste stable et utilisable en conditions réelles (en production).

Aujourd'hui, EduTrack est une plateforme fonctionnelle qui montre concrètement comment la Data et l'Intelligence Artificielle peuvent être utilisées pour résoudre des problèmes humains, comme la prévention du décrochage scolaire.

Merci !!